

Agentic Role Bounding and Optimization in Multi-Agent LLM Systems for Software Engineering

1st Shahab Ali

Faculty of Artificial Intelligence Technologies (FATII)
ITMO University
Saint Petersburg, Russia
Group J4132, Student ID: 503271

2nd Sergey Kovalchuk

Faculty of Artificial Intelligence Technologies (FATII)
ITMO University
Saint Petersburg, Russia

Abstract—This paper develops a methodological framework for defining, selecting, and measuring agent roles in multi-agent large language model systems for software engineering. The work builds on a structured review of 19 sources and a two-phase empirical study comprising 937 runs. An initial exploratory phase examined framework behavior and failure modes, while the main evaluation compared four role configurations across HumanEval and SWE-bench Lite using the same underlying model. A 140-run prompt ablation was then used to examine which prompt components contribute most strongly to role effectiveness.

The results motivate two research directions. First, we propose the *Constrained Action Space* as an operational definition of an agent role, where the role is characterized by permitted actions, output constraints, and tool restrictions rather than semantic identity alone. In the ablation study, identity-only and identity-plus-responsibility prompts failed on all 20 tested HumanEval tasks, whereas variants containing a strict output-format constraint succeeded on 19/20 or 20/20 tasks. These results provide evidence, within the tested setting, that operational boundaries contribute substantially more to successful execution than role labels alone. Role optimization is then formulated as a state-driven selection problem. A prototype decision-tree selector achieves 94.53% cross-validated accuracy, below the 95.13% majority-class baseline, indicating that the evaluated benchmark contains too few architecture-sensitive tasks for effective learned selection.

Second, we propose architecture-level measurements that assess system behavior independently of final task success: *Coordination Friction*, *Autonomous Recovery Rate*, and *Boundary Violation Rate*. *Autonomous Recovery Rate* is measured at 0.90 from exploratory observations, while *Coordination Friction* and *Boundary Violation Rate* remain proposed metrics requiring further instrumentation. The study therefore shifts the focus from static role labels toward controllable role boundaries, conditional role selection, and direct measurement of agent architecture behavior.

Index Terms—multi-agent systems, large language models, software engineering, agent roles, role optimization, constrained action space, agent orchestration, OpenHands

I. INTRODUCTION

Large language models are increasingly being used as autonomous software engineering agents capable of planning, generating code, executing tools, and interacting with development environments. Multi-agent systems extend this idea

by assigning different responsibilities to specialized agents such as managers, coders, and testers. Frameworks including MetaGPT [1], ChatDev [2], and Magentic-One [3] demonstrate that such role-based architectures can support complex software-development workflows.

However, two fundamental questions remain insufficiently addressed. First, agent roles are commonly represented through semantic identities, job titles, or natural-language responsibilities, but these descriptions do not formally specify what an agent is permitted or forbidden to do. Second, role architectures are often selected statically, even though additional agents may introduce communication, execution, and coordination overhead that is not beneficial for every task.

This study develops from an earlier structured review of 19 sources conducted during the first phase of the research. That review produced a seven-layer taxonomy of agent roles covering Orchestration, Analysis and Planning, Design, Implementation, Quality Assurance, Knowledge and Documentation, and Operations. The taxonomy identified what kinds of roles are commonly used, but it also exposed a limitation: existing role descriptions primarily explain what a role is called or expected to do, rather than how the role should be operationally bounded.

The research therefore shifts from descriptive role taxonomy toward methods for role definition, optimization, and measurement. The study is organized around two research questions:

RQ1 — Role Exposure and Optimization: How can an agent role be formally defined so that its effect is controllable, and how can an appropriate role configuration be selected according to task state?

RQ2 — Architecture Measurement: How can the intrinsic efficiency and stability of an agent-role architecture be measured independently of final task success?

To address these questions, the study combines prior taxonomy work, exploratory OpenHands experiments, a large-scale controlled evaluation, a prompt ablation study, and a prototype

state-prediction model. The full empirical program comprises 937 runs: 21 exploratory Phase 1 runs, 656 HumanEval role-configuration evaluations, 120 SWE-bench Lite evaluations, and 140 prompt-ablation runs.

The main contributions are:

- 1) **Structured role taxonomy as motivating foundation.** A structured review of 19 sources produced a seven-layer taxonomy of agent roles and identified Quality Assurance as the most frequently represented role family. This prior phase motivates the transition from semantic role descriptions toward operational role definitions.
- 2) **Constrained Action Space for role exposure.** We propose defining an agent role through explicit operational boundaries rather than semantic identity alone. A role is represented by permitted actions, output constraints, and tool restrictions. A 140-run ablation study provides empirical support for this formulation in the tested setting.
- 3) **State-driven role optimization.** We formulate role selection as a task-state-to-architecture mapping problem. A prototype decision-tree selector is evaluated on 164 HumanEval tasks. It achieves 94.53% cross-validated accuracy compared with a 95.13% majority-class baseline, providing a negative result that reveals a strong ceiling effect caused by the small number of architecture-sensitive tasks.
- 4) **Architecture-level measurement.** We propose three log-based measurements: Coordination Friction, Autonomous Recovery Rate, and Boundary Violation Rate. Autonomous Recovery Rate is measured at 0.90 from Phase 1 observations, while the remaining metrics define instrumentation targets for future evaluation.
- 5) **Empirical evidence on role-dependent overhead.** On the evaluated SWE-bench Lite subset, the Coder+Tester configuration is 99.2 seconds slower per task on average than the single-Coder configuration. Although resolve-rate differences are not statistically significant, the timing results motivate explicit measurement of coordination cost.

The goal of the paper is therefore not to identify a universally best agent topology, but to develop a more general framework for understanding how agent roles should be bounded, when different role architectures should be selected, and how the architecture itself can be measured.

II. BACKGROUND AND RELATED WORK

A. Multi-Agent LLM Systems for Software Engineering

Recent multi-agent LLM systems organize software-development tasks through specialized agents and structured interaction patterns. MetaGPT [1] represents software-development roles through Standard Operating Procedures (SOPs), assigning responsibilities to agents such as Product Manager, Architect, Engineer, and QA Engineer. ChatDev [2] structures collaboration through a Chat Chain in which agents

occupy different positions in a software-development workflow.¹ Magentic-One [3] employs a Manager-Worker architecture in which an orchestrating agent delegates work to specialist agents.

Other systems explore different forms of specialization and execution. PotPie [4] grounds agent behavior in repository knowledge representations, while MapCoder [5] separates planning, implementation, and debugging for code-generation tasks. OpenHands [6] provides an execution-oriented environment in which agents can manipulate files, invoke tools, and execute code. In contrast, Agentless [7] demonstrates that repository-level software repair can sometimes be performed effectively without an explicit multi-agent architecture. Recent surveys [8] further catalogue multi-agent applications across the software-development lifecycle.

These systems show that role specialization is widely used, but role definitions are commonly expressed through names, responsibilities, prompts, or workflow positions. Such semantic definitions describe what an agent is expected to do, but they do not necessarily specify the operational boundaries that determine which actions, tools, or outputs are permitted.

B. Prior Research Phase: Agent Role Taxonomy

The present study builds on a completed prior research phase conducted during Semester 1. A structured review of 19 sources was performed using PRISMA-style inclusion and exclusion criteria. The review included seven multi-agent systems together with twelve surveys, taxonomies, and empirical studies.

From this material, a seven-layer agent-role taxonomy was constructed:

- 1) Orchestration,
- 2) Analysis and Planning,
- 3) Design,
- 4) Implementation,
- 5) Quality Assurance,
- 6) Knowledge and Documentation,
- 7) Operations.

Quality Assurance was the most frequently represented role family, appearing in nine of the 19 reviewed sources, while Design appeared least frequently. This result suggested that contemporary multi-agent software-engineering systems place substantial emphasis on verification and feedback in addition to code generation.

The taxonomy was an original synthesis produced during the first research phase and served as the conceptual foundation for the later experimental work. In particular, the prevalence of Quality Assurance motivated explicit evaluation of Tester roles, while the observation that existing frameworks primarily describe roles semantically motivated the later investigation of operational role boundaries.

The taxonomy therefore remains an important part of this study, but it is not retained as a separate final research question.

¹ChatDev is discussed here as additional related work; the structured taxonomy analysis was based on the 19 sources included in the completed review.

Its purpose in the final structure is to establish what kinds of agent roles exist and to motivate the transition toward methods for formally defining and optimizing them.

C. From Semantic Roles to Operational Role Boundaries

Existing multi-agent systems typically associate a role with one or more of the following elements: a semantic identity or title, a natural-language responsibility, a workflow position, a Standard Operating Procedure, or access to particular tools or resources. However, these components are often combined within a single prompt or system design, making it difficult to determine which component is responsible for observed performance differences.

This motivates the first part of RQ1. Rather than treating role names as the primary unit of analysis, this study investigates whether a role can be represented through explicit operational constraints on the agent’s available actions, outputs, and tools. The later prompt-ablation experiment was designed specifically to separate semantic identity, responsibility descriptions, and output-format constraints. The resulting evidence forms the empirical basis for the Constrained Action Space formulation introduced in Section IV.

D. Static and Adaptive Agent Architectures

Role-based architectures are frequently configured before execution. A fixed architecture may include a single coder, a coder and tester, or a larger hierarchical team. However, adding agents can introduce additional communication and execution cost. This creates a second problem within RQ1: the architecture appropriate for one task may not be appropriate for another.

The present work therefore considers role selection as a conditional decision problem in which characteristics of the task state are used to select among available role configurations. The study evaluates an initial supervised implementation of this idea using a decision-tree classifier. The objective is not to claim that the decision tree itself is the final orchestration method, but to test whether task-level features contain sufficient information to support architecture selection.

E. Evaluating Agent Architectures Beyond Final Accuracy

Most evaluations of software-engineering agents rely primarily on task-level outcome metrics such as pass@1, resolve rate, or successful patch generation. These metrics are necessary, but they do not directly measure how the architecture behaves internally. Two systems may achieve the same final outcome while differing substantially in coordination effort, error-recovery behavior, adherence to assigned role boundaries, or execution overhead.

This motivates RQ2. The study therefore proposes architecture-level measurements derived from execution logs rather than final output alone: Coordination Friction, Autonomous Recovery Rate, and Boundary Violation Rate. Autonomous recovery is partially observable in the exploratory Phase 1 data, while Coordination Friction and Boundary Violation Rate remain proposed measurements requiring additional instrumentation.

III. EMPIRICAL FOUNDATION AND EXPERIMENTAL METHODOLOGY

The methodological framework proposed in this paper is grounded in three empirical components: an exploratory Phase 1 study, a controlled Phase 2 role-configuration evaluation, and a prompt-ablation study. Together, these components comprise 937 experimental runs.

A. Overall Experimental Program

TABLE I
EXPERIMENTAL PROGRAM SUMMARY

Experiment	Design	Runs
Phase 1 HumanEval	15 exploratory	15
Phase 1 SWE-bench	6 exploratory	6
Phase 2 HumanEval	164 × 4	656
Phase 2 SWE-bench Lite	30 × 4	120
Prompt Ablation	20 × 7	140
Total		937

The experiments were conducted using the OpenHands CodeAct framework, which provides an execution environment in which the model can reason, invoke tools, manipulate files, and execute code.

B. Phase 1: Exploratory Framework Evaluation

Phase 1 consisted of 21 exploratory runs: 15 HumanEval runs and six SWE-bench runs. The exact allocation of Phase 1 HumanEval tasks across model sizes and role configurations cannot be reconstructed reliably because this phase predated the structured logging used later in the study. The results were recorded primarily through weekly research reports and qualitative observations.

Accordingly, Phase 1 is not treated as a controlled benchmark comparison. Its purpose in the present paper is to document phenomena that motivated the later experimental design. The safely recoverable observations include difficulties deploying smaller local models within the CodeAct tool-calling environment, repeated XML schema and execution failures, observations later described as Agentic Paralysis, autonomous recovery from nine of ten recorded error events, and context or token exhaustion in prolonged failure loops.

Because no structured event log was retained for Phase 1, the nine-out-of-ten recovery count should be interpreted as a qualitative research record rather than as a fully instrumented benchmark result.

C. Phase 2: Controlled HumanEval Evaluation

The main controlled evaluation uses the same underlying model, `claude-haiku-4-5`, for all role configurations. This removes model identity as an experimental factor and allows the role configuration itself to be compared more directly.

The complete HumanEval benchmark [9] of 164 tasks is evaluated under four configurations: Baseline (no explicit role

architecture), `Coder`, `Coder+Tester`, and `Full System (Manager+Coder+Tester)`. This produces $164 \times 4 = 656$ unique HumanEval task–configuration evaluations.

The HumanEval execution timeout was set to 400 seconds. Generated code was evaluated through in-process execution together with the benchmark `check()` function. The generation temperature and LLM execution random seed were not recorded. The Docker environment used the unpinned image `ghcr.io/all-hands-ai/openhands:latest`, which limits exact reproducibility.

D. Phase 2: SWE-bench Lite Evaluation

To examine repository-level behavior, the same four configurations were evaluated on the first 30 tasks from SWE-bench Lite [10], producing $30 \times 4 = 120$ runs. The timeout was 600 seconds per task.

Unlike the official SWE-bench evaluation procedure, resolution in this experiment was determined using file-level overlap between the generated Git diff and the reference patch. Therefore, the reported resolve rates should not be interpreted as official SWE-bench scores. This evaluation remains useful for comparing the four configurations under a consistent internal criterion, particularly with respect to runtime and timeout behavior. All recorded failures in this subset were timeout failures; no wrong-patch or rejected-patch category was recorded under the internal evaluation criterion.

E. Prompt Ablation Study

The prompt-ablation study examines which components of the role prompt are associated with successful execution. The first 20 HumanEval tasks were evaluated under seven prompt variants, producing $20 \times 7 = 140$ runs.

The variants separate three prompt components: semantic role identity, responsibility description, and explicit output-format constraint. The prompts were not token- or length-matched. Therefore, the experiment does not fully control prompt length. However, the short constraint-only prompt succeeds on 19 of 20 tasks, while a longer unconstrained multi-role prompt fails on all 20 tasks. This makes a simple explanation based only on prompt length unlikely, without claiming that prompt length has been completely ruled out.

F. State-Prediction Experiment

To investigate whether role configuration can be selected from task characteristics, each HumanEval prompt is represented by a 12-dimensional feature vector: number of lines, number of words, number of characters, return-hint count, list-type count, dictionary-type count, optional-type count, docstring-block count, example count, indentation count, count of words longer than eight characters, and primitive type-token count.

A `DecisionTreeClassifier` with maximum depth four and `random_state=42` is evaluated using stratified five-fold cross-validation with shuffling enabled. Role-selection labels are generated using the first successful configuration in the fixed order `Baseline` $\rightarrow$ `Coder` $\rightarrow$ `Coder+Tester`

$\rightarrow$ `Full System`. Tasks solved by no configuration are assigned to the `Baseline` class.

This produces a highly imbalanced label distribution: 156 `Baseline` tasks and eight `Coder` tasks, with no `Coder+Tester` or `Full System` labels under this rule.

G. Statistical and Reproducibility Considerations

Pairwise SWE-bench configuration comparisons were examined using McNemar tests on paired task outcomes. None of the six pairwise comparisons reached statistical significance at the conventional $p < 0.05$ threshold.

For the state-prediction experiment, the five cross-validation scores are 0.9394, 0.9394, 0.9697, 0.9091, and 0.9688, giving a mean accuracy of 94.53%. The majority-class policy achieves 95.13%. Therefore, the learned decision tree does not outperform the trivial policy. The present study reports this negative result explicitly rather than interpreting near-equal performance as evidence of successful dynamic selection.

IV. RQ1: ROLE EXPOSURE AND OPTIMIZATION

RQ1 asks how an agent role can be formally defined so that its effect is controllable, and how an appropriate role configuration can be selected according to task state. The answer developed in this study has two parts: role exposure through a Constrained Action Space and role optimization through state-driven selection.

A. Role Exposure Through a Constrained Action Space

Existing multi-agent systems commonly represent roles through semantic identities such as “`Coder`,” “`Tester`,” or “`Manager`,” often combined with natural-language responsibilities or workflow positions. However, such labels do not formally specify the operational boundaries of the role.

We therefore propose representing a role by the subset of actions, outputs, and tools that are permitted under that role. Let $\mathcal{A}$ denote the unconstrained action space available to the underlying agent. For a role R and task state s , we define the role-specific action space as

$$\mathcal{A}_R(s) \subseteq \mathcal{A}. \quad (1)$$

Operationally, the role is represented as

$$R = (\mathcal{A}_R, \mathcal{O}_R, \mathcal{T}_R), \quad (2)$$

where $\mathcal{A}_R$ denotes the permitted action set, $\mathcal{O}_R$ denotes output-format or schema constraints, and $\mathcal{T}_R$ denotes permitted tools or tool transitions. We refer to this representation as the *Constrained Action Space*. Under this formulation, a semantic title is not itself sufficient to define a role. A role becomes operational when the system specifies which behaviors are permitted and which behaviors constitute violations.

B. Prompt Ablation Evidence

The prompt-ablation experiment provides the main empirical evidence for this formulation.

TABLE II
PROMPT ABLATION RESULTS

Variant	Constraint	Pass@1
Control	Yes	95.0%
Coder A	No	0.0%
Coder B	No	0.0%
Coder C	Yes	100.0%
Full A	No	0.0%
Full B	No	0.0%
Full C	Yes	95.0%

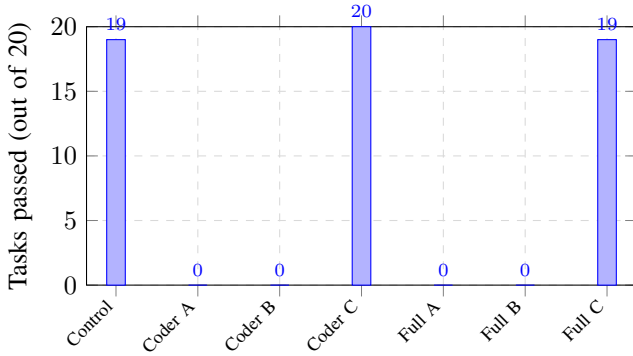


Fig. 1. Prompt ablation results on 20 HumanEval tasks. Only variants containing the explicit output-format constraint (Control, Coder C, and Full C) achieve successful executions.

Coder A contains identity only; Coder B adds a responsibility description; Coder C adds the output-format constraint. Full A contains multi-role identities only; Full B adds responsibilities; Full C adds the final output-format constraint.

The separation between constrained and unconstrained variants is substantial. Identity-only and identity-plus-responsibility conditions fail on all 20 tasks, while all three variants containing a strict output-format constraint succeed on 19 or 20 tasks. The constraint-only control succeeds on 19 of 20 tasks without any semantic role identity. This provides evidence that the operational output boundary contributes substantially more to successful execution than the role label itself in this experimental setting.

The observed failure mechanism also supports this interpretation. In unconstrained variants, the model frequently produces explanatory prose or Markdown around the function body, causing the evaluation harness to reject or fail to parse the generated output. However, the ablation prompts were not token- or length-matched. The short constraint-only prompt succeeds on 19/20 tasks while the longer unconstrained Full B prompt fails on 0/20 tasks, making a simple prompt-length explanation unlikely. Nevertheless, the experiment does not fully control prompt length and should not be interpreted as a complete causal decomposition of all prompt effects.

C. Interpretation of the Role-Bounding Result

The ablation does not establish that semantic role descriptions are universally useless. Instead, it demonstrates that, in this setting, semantic identity alone is insufficient to produce reliable executable output, while a strict operational output boundary is strongly associated with success.

The Constrained Action Space generalizes the experimental finding beyond output formatting. Output constraints represent one type of boundary. Other possible boundaries include permitted tool sets, file-system access restrictions, action-transition constraints, communication permissions, schema restrictions, and stopping conditions. These additional forms of role bounding are proposed extensions and were not independently evaluated in the present study.

D. Role Optimization as State-Driven Selection

The second part of RQ1 concerns role optimization. The Phase 2 results indicate that additional roles are not uniformly beneficial across tasks.

TABLE III
HUMANEVAL ROLE-CONFIGURATION RESULTS

Configuration	Pass	Pass@1
Baseline	151/164	92.1%
Coder	155/164	94.5%
Coder+Tester	152/164	92.7%
Full System	156/164	95.1%

On HumanEval, the full Manager+Coder+Tester configuration produces the highest observed pass@1. On the selected SWE-bench Lite subset, the single-Coder configuration produces both the highest observed resolve rate and the lowest mean runtime. This motivates treating architecture selection as a conditional decision rather than a fixed design choice.

Let $s \in \mathcal{S}$ denote the state representation of a software task and $a \in \mathcal{C}$ denote one available role configuration, where

$$\mathcal{C} = \{\text{baseline, coder, coder+tester, full system}\}. \quad (3)$$

The role-optimization problem is to learn a policy

$$\pi(a \mid s) \quad (4)$$

that selects the configuration most appropriate for the current task. The present study implements a supervised prototype using a decision-tree classifier.

E. Prototype Selector Results

The fold accuracies are 0.9394, 0.9394, 0.9697, 0.9091, and 0.9688, yielding a mean cross-validated accuracy of 94.53%. The trivial majority-class policy achieves 95.13%. The learned selector therefore performs 0.61 percentage points worse than the trivial policy.

The selector result should therefore be interpreted as a negative result rather than a near-success.

TABLE IV
STATE-DRIVEN SELECTOR PERFORMANCE

Method	Accuracy
Majority-class policy	95.13%
Decision Tree selector	94.53%
Difference	-0.61 pp

F. Why the Learned Selector Does Not Improve Performance

Of the 164 HumanEval tasks, 145 produce identical outcomes across all four configurations and 19 are architecture-sensitive. Only eight tasks are rescued by a role configuration after the baseline fails, while 11 baseline successes are lost by at least one role configuration. Five tasks fail under every configuration: HumanEval/32, HumanEval/38, HumanEval/50, HumanEval/116, and HumanEval/145.

The maximum theoretical selector success rate is therefore

$$\frac{164 - 5}{164} = \frac{159}{164} = 96.95\%. \quad (5)$$

The baseline-dominated label distribution leaves only a narrow range between the trivial majority policy and the theoretical maximum. This creates a ceiling effect that limits what a supervised selector can learn from this benchmark.

G. Observed Coordination Overhead

The SWE-bench Lite results provide an additional motivation for state-dependent role selection.

TABLE V
SWE-BENCH LITE TIMING RESULTS

Configuration	Resolve Rate	Mean Time
Baseline	73.3%	296.0 s
Coder	83.3%	232.1 s
Coder+Tester	66.7%	331.3 s
Full System	80.0%	295.2 s

The Coder+Tester configuration is $331.3 - 232.1 = 99.2$ seconds slower per task on average than the single-Coder configuration. All six pairwise McNemar tests on SWE-bench task outcomes have $p > 0.05$, so the resolve-rate differences should not be treated as statistically established. The timing difference is the more defensible empirical signal.

These results motivate the broader principle underlying state-driven selection: additional agents should be deployed only when their expected benefit exceeds their coordination and execution cost.

H. Answer to RQ1

The evidence supports a two-part answer to RQ1. First, an agent role can be exposed operationally as a Constrained Action Space rather than only as a semantic identity. The prompt-ablation study shows that explicit output boundaries are strongly associated with successful execution in the tested setting. Second, role optimization can be formulated as state-driven selection over alternative architectures. The prototype

decision tree does not beat the trivial baseline because the evaluated HumanEval benchmark is dominated by tasks whose outcomes do not depend on the architecture. More expressive policies, including reinforcement-learning approaches, remain future work on benchmarks with greater architecture sensitivity.

V. RQ2: ARCHITECTURE MEASUREMENT

RQ2 asks how the intrinsic efficiency and stability of an agent-role architecture can be measured independently of final task success. Outcome metrics such as pass@1 and resolve rate are essential for measuring whether a task was completed successfully, but they do not describe how the architecture behaved internally. Two architectures can produce the same final result while differing substantially in coordination effort, recovery behavior, or adherence to assigned role boundaries.

To address this limitation, we propose three log-based architecture metrics: Coordination Friction, Autonomous Recovery Rate, and Boundary Violation Rate.

A. Coordination Friction

The SWE-bench Lite results indicate that additional role structure can introduce measurable execution overhead. Runtime alone, however, cannot distinguish whether the additional time was spent on useful computation, redundant communication, or repeated coordination.

Let T_{coord} denote the number of tokens associated with coordination-related events, such as inter-agent messages and other events whose primary purpose is to coordinate work among agents. Let T_{total} denote the total token usage of the complete run. Coordination Friction is defined as

$$F_{\text{coord}} = \frac{T_{\text{coord}}}{T_{\text{total}}}. \quad (6)$$

A value close to zero indicates that little of the execution budget is consumed by explicit coordination, whereas a larger value indicates that a greater proportion of the context is devoted to communication and coordination.

This metric is proposed but not yet computed in the present study. The current experimental logs record elapsed execution time but do not contain sufficiently detailed token-level event instrumentation. The 99.2-second SWE-bench timing difference is therefore treated only as a motivation and proxy observation, not as a measured value of F_{coord} .

B. Autonomous Recovery Rate

The exploratory Phase 1 experiments revealed repeated cases in which the agent encountered an execution or framework error and subsequently recovered without human intervention.

Let E_{total} denote the number of observed recoverable error events and $E_{\text{recovered}}$ denote the number of those events followed by successful autonomous recovery. We define

$$R_{\text{auto}} = \frac{E_{\text{recovered}}}{E_{\text{total}}}. \quad (7)$$

In Phase 1, nine of ten recorded error events were followed by autonomous recovery:

$$R_{\text{auto}} = \frac{9}{10} = 0.90. \quad (8)$$

A recovery event was recorded when an error such as an XML schema failure, file conflict, or execution error was followed by a corrective agent action and successful continuation of the task without external human intervention. This result should be interpreted cautiously because Phase 1 did not use structured automated event logging and the ten events were reconstructed from qualitative weekly research reports.

C. Boundary Violation Rate

The Constrained Action Space formulation implies that a role should be evaluated not only by whether the task succeeds, but also by whether the agent remains within the role’s permitted operational boundary.

Let N_{viol} denote the number of observed actions or outputs that violate the role specification and let N_{role} denote the total number of evaluated role-governed actions. Boundary Violation Rate is defined as

$$V_{\text{boundary}} = \frac{N_{\text{viol}}}{N_{\text{role}}}. \quad (9)$$

Candidate violation events include malformed output schemas, prohibited output formats, unauthorized tool use, execution outside the role’s permitted action set, communication outside permitted role transitions, and failure to obey stopping or transition conditions.

The Phase 1 failure taxonomy provides candidate event types that may be used to operationalize this metric. Examples include XML schema errors, system-prompt bleed, and placeholder hallucination. However, the current study did not implement an automated boundary-violation counter. Therefore, V_{boundary} remains a proposed metric.

D. Relationship Between the Three Metrics

TABLE VI
PROPOSED ARCHITECTURE-LEVEL METRICS

Metric	Property	Status
F_{coord}	Coordination overhead	Proposed
R_{auto}	Error recovery	Exploratory
V_{boundary}	Role adherence	Proposed

Together, these metrics separate three questions that final task accuracy cannot answer: how much of the system’s computational effort is spent on coordination, how effectively the architecture can recover from intermediate failures, and how reliably the architecture respects the operational boundaries assigned to its roles.

E. Log-Based Measurement Framework

The proposed metrics are intended to be computed from structured agent event logs rather than inferred only from final outputs. A future instrumentation pipeline would associate each event with a timestamp, agent identity, event type, token count, tool invocation, execution result, error status, recovery status, and role-boundary status. This would allow the architecture to be evaluated as a dynamic system rather than only as a source of final answers.

F. Answer to RQ2

RQ2 is answered by proposing a log-based measurement framework for intrinsic architecture behavior. Coordination Friction measures the relative cost of coordination, Autonomous Recovery Rate measures the ability of the architecture to recover from intermediate failures, and Boundary Violation Rate measures adherence to operational role boundaries. Of the three metrics, Autonomous Recovery Rate is directly estimated in the present study at 0.90 from exploratory Phase 1 observations. Coordination Friction and Boundary Violation Rate remain proposed metrics that require structured event-level instrumentation before they can be evaluated empirically.

VI. DISCUSSION

The results of this study suggest that agent roles in multi-agent LLM systems should be understood as more than semantic labels. The empirical evidence indicates that operational constraints can have a stronger effect on successful execution than role identity alone, at least within the tested HumanEval setting.

The prompt-ablation study provides the clearest evidence for this point. Identity-only and identity-plus-responsibility prompts fail on all 20 tasks, whereas prompt variants containing an explicit output-format constraint succeed on 19 or 20 tasks. This supports the Constrained Action Space formulation introduced under RQ1. However, the result should not be interpreted as evidence that semantic roles are universally irrelevant. The ablation evaluates one model, one benchmark subset, and one principal type of constraint.

The second part of RQ1 concerns task-dependent role selection. The controlled HumanEval and SWE-bench evaluations show that additional role structure is not uniformly beneficial. The full Manager+Coder+Tester configuration produces the highest observed HumanEval pass@1, while the single-Coder configuration performs best on the selected SWE-bench Lite subset in both observed resolve rate and mean runtime.

The decision-tree selector was introduced to test whether such differences could be predicted from task characteristics. Its 94.53% cross-validated accuracy does not exceed the 95.13% majority-class baseline. This negative result is informative rather than unsuccessful. The benchmark is dominated by architecture-insensitive tasks: 145 of 164 HumanEval tasks produce identical outcomes across all four configurations, and only eight tasks are rescued by a role configuration after baseline failure.

Of the eight tasks rescued after baseline failure, all eight are solved by the Coder configuration, and seven of the eight are also solved by both Coder+Tester and Full System. Only HumanEval/10 produces a different outcome: Coder and Full System pass while Coder+Tester fails. This suggests that on these tasks, additional roles did not add recovery capability beyond what the single-Coder configuration already provided.

Consequently, the present benchmark provides only a narrow margin in which learned orchestration can improve over a trivial policy. This suggests that future dynamic-selection experiments should emphasize benchmarks with a larger proportion of architecture-sensitive tasks rather than relying primarily on benchmarks already solved reliably by the baseline configuration.

The SWE-bench timing results reinforce this interpretation. Adding a Tester to the Coder increases average execution time by 99.2 seconds per task. Because none of the six pairwise McNemar comparisons reaches statistical significance, the observed resolve-rate differences should not be treated as established performance advantages or disadvantages. The runtime difference is the stronger empirical indication that additional agents may create measurable coordination or execution cost.

This observation motivates RQ2. Final task success alone cannot distinguish between architectures that arrive at the same answer through different internal processes. A system may solve a task successfully while consuming unnecessary coordination effort, repeatedly violating role boundaries, or recovering from multiple intermediate failures.

The proposed architecture-level metrics are intended to expose these differences. Only Autonomous Recovery Rate is directly estimated in the current study. The Phase 1 observations yield $R_{\text{auto}} = 0.90$, but this value is reconstructed from qualitative weekly reports rather than structured event logs. Coordination Friction and Boundary Violation Rate remain proposed measurements whose contribution in the present work is methodological.

The overall framework that emerges from the study can be summarized as

$$\begin{aligned} \text{Role Taxonomy} &\rightarrow \text{Role Bounding} \\ &\rightarrow \text{State-Driven Selection} \\ &\rightarrow \text{Architecture Measurement.} \end{aligned} \quad (10)$$

The taxonomy identifies the functional roles that appear in existing systems. The Constrained Action Space specifies how those roles can be operationally bounded. State-driven selection addresses when different role configurations should be used. Architecture-level metrics provide a way to evaluate whether the selected architecture operates efficiently and remains within its assigned boundaries.

The study therefore does not claim to identify a universally optimal multi-agent architecture. Its main contribution is a methodological shift: from asking which named role configuration performs best toward asking how roles should be bounded, when architectures should be selected, and how their internal behavior should be measured.

VII. LIMITATIONS

The study has several limitations that affect the strength and generalizability of its conclusions.

- 1) **Single execution per Phase 2 condition.** Each HumanEval and SWE-bench task-configuration pair was executed once, so repeated-run variance estimates are unavailable.
- 2) **Incomplete generation reproducibility.** The generation temperature and LLM execution random seed were not recorded. The OpenHands environment also used the unpinned Docker image `ghcr.io/all-hands-ai/openhands:latest`. The `random_state=42` used in the later decision-tree analysis applies only to the machine-learning selector, not to the original LLM executions.
- 3) **Exploratory nature of Phase 1.** The exact allocation of the 21 Phase 1 runs across model sizes and role configurations cannot be reconstructed reliably. Phase 1 is therefore used only as a source of qualitative observations and methodological motivation.
- 4) **Qualitative autonomous-recovery evidence.** The value $R_{\text{auto}} = 9/10 = 0.90$ was reconstructed from weekly research reports rather than structured event logs.
- 5) **Non-random benchmark subsets.** The SWE-bench Lite evaluation uses the first 30 tasks and the prompt ablation uses the first 20 HumanEval tasks rather than randomized samples.
- 6) **Internal SWE-bench resolution criterion.** Repository-level resolution was evaluated using file-level Git diff overlap rather than the official SWE-bench test-suite procedure; reported resolve rates should not be interpreted as official SWE-bench scores.
- 7) **Small ablation sample.** The prompt ablation contains 20 HumanEval tasks for each of seven variants and should be replicated on larger task sets and additional benchmarks.
- 8) **Prompt length not fully controlled.** The ablation variants were not token- or character-length matched. A simple length explanation is unlikely but not fully eliminated.
- 9) **Limited constraint types.** The empirical evidence for the Constrained Action Space primarily evaluates output-format constraints. Tool restrictions, communication restrictions, and action-transition rules were not independently evaluated.
- 10) **Limited model generalization.** The controlled Phase 2 and ablation experiments use `claude-haiku-4-5`; replication across additional model families is required.
- 11) **Highly imbalanced selector labels.** The state-selection dataset contains 156 Baseline labels and eight Coder labels, with no Coder+Tester or Full System labels under the adopted labelling rule.
- 12) **Label-generation bias.** When multiple configurations solve the same task, the first successful configuration in the fixed order receives the label, systematically favoring

simpler configurations.

- 13) **Limited architecture-sensitive tasks.** Only 19 of the 164 HumanEval tasks produce different outcomes across configurations, and only eight are rescued after baseline failure.
- 14) **Partially unvalidated architecture metrics.** Coordination Friction and Boundary Violation Rate have not yet been computed because the current logs lack the required event-level instrumentation.

These limitations restrict the scope of the conclusions but also define a clear validation agenda: replication across models and benchmarks, controlled experiments over multiple boundary types, repeated stochastic runs, official repository-level evaluation, and structured event-log instrumentation.

VIII. CONCLUSION AND FUTURE WORK

This study develops a methodological framework for understanding agent roles in multi-agent LLM systems for software engineering. Rather than treating roles only as semantic identities or fixed positions in a workflow, the study focuses on three linked problems: how roles should be operationally bounded, how role architectures should be selected according to task state, and how the architecture itself should be measured.

The research builds on a completed seven-layer agent-role taxonomy and a 937-run empirical program comprising exploratory framework observations, controlled HumanEval and SWE-bench Lite evaluations, and a prompt ablation study.

For RQ1, the study proposes the *Constrained Action Space* as an operational representation of an agent role. The prompt-ablation results show that semantic identity and responsibility descriptions alone fail on all 20 tested HumanEval tasks, while variants containing a strict output-format constraint succeed on 19 or 20 tasks. These findings provide empirical support, within the tested setting, for treating operational boundaries as a central component of role definition.

Role optimization is then formulated as a state-driven selection problem. A decision-tree selector trained on task features achieves 94.53% cross-validated accuracy, below the 95.13% majority-class baseline. This negative result reveals a strong ceiling effect: most HumanEval tasks are insensitive to the role architecture, leaving too few architecture-sensitive cases for a learned selector to exploit.

For RQ2, the study proposes three architecture-level measurements: Coordination Friction, Autonomous Recovery Rate, and Boundary Violation Rate. Autonomous Recovery Rate is estimated at 0.90 from the exploratory Phase 1 observations. Coordination Friction and Boundary Violation Rate remain proposed metrics requiring structured event-level instrumentation.

The broader contribution of the work is therefore not a claim that one multi-agent topology is universally superior. Instead, the study proposes a shift from static role naming toward a more systematic view of agent architecture:

Role Taxonomy → Role Bounding
→ State-Driven Selection (11)
→ Architecture Measurement.

Future work should first replicate the Constrained Action Space ablation across additional models, larger task sets, and multiple constraint types, including tool restrictions, communication permissions, and action-transition rules. Second, repository-level evaluation should be repeated using the official SWE-bench test suite and repeated stochastic runs. Third, OpenHands event logging should be extended to record per-event token usage, agent identity, tool calls, recovery events, and boundary violations so that Coordination Friction and Boundary Violation Rate can be computed directly. Finally, state-driven orchestration should be evaluated on benchmarks containing a larger proportion of architecture-sensitive tasks before more complex policies, including reinforcement-learning approaches, are introduced.

REFERENCES

- [1] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *Proc. ICLR*, 2024.
- [2] C. Qian et al., “ChatDev: Communicative Agents for Software Development,” in *Proc. ACL*, 2024.
- [3] A. Fournay et al., “Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks,” arXiv:2411.04468, 2024.
- [4] PotPie AI, “PotPie: Custom Engineering Agents,” GitHub repository, 2025. [Online]. Available: <https://github.com/potpie-ai/potpie>
- [5] M. A. Islam et al., “MapCoder: Multi-Agent Code Generation for Competitive Problem-Solving,” in *Proc. ACL*, 2024.
- [6] X. Wang et al., “OpenHands: An Open Platform for AI Software Developers,” arXiv:2407.16741, 2024.
- [7] C. Xia et al., “Agentless: Demystifying LLM-based Software Engineering Agents,” arXiv:2407.01489, 2024.
- [8] Z. Rasheed et al., “LLM-Based Multi-Agent Systems for Software Engineering,” arXiv:2404.04834, 2024.
- [9] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.
- [10] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in *Proc. ICLR*, 2024.